

JC Williams

2/1/26

CS-401

Prof. Unan

HW1

The two implementations of the inventory system, C++ and Python, varied dramatically in several different ways including ease of implementation, readability and maintainability, handling of object-oriented concepts, and error handling. While both languages provided functionally identical outputs the development process revealed distinct trade-offs. In general Python performed better in terms of implementation and readability while both Python and C++ were a mixed bag with object oriented (OOP) concepts and error handling.

The ease of implementation is one of the largest factors when considering a programming language for a smaller project like the inventory system. This is because performance isn't a big factor when both can run with minimal resources. Python easily outperformed C++ with a much faster development cycle. Being a dynamic language features like list and object types could be easily used without worrying about memory management and strict type definitions. Python required no setup files and could run with a single script or a simple folder. Python's syntax is also simpler with operations like f-strings and list comprehension allowing for complex filtering logic to be implemented in just a single line of code. This is unlike C++ which requires boilerplate code to machine the same results. The setup with C++ was more complex with extra files like the Makefile and headers being necessary to build the process. Additionally C++ required manual memory allocation for objects like needing pointers (Item*) to store child

classes in a vector. It also required destructors to prevent memory leaks which is completely unnecessary in Python. Python is much better for a rapid implementation while C++ requires a lot of finesse with memory and linking.

Python also excels with readability as it is intended to. Python's syntax prioritises conciseness, a striking example is the `display_filtered_items` method which is roughly half the lines of the C++ counterpart. And as previously stated the file structure is simpler, not needing header files keeps with file count low and easier to maintain. This is a double edged sword however because while analyzing the C++ implementation the reader only needs to look at the header files to know what `Item` does for example. This lessens the burden of having to read through the source code and complex implementation of each object. But in the end C++ is noisier than python, needing explicit types like `std::vector<T>` and `std::string` which makes the code harder to interpret at a glance. In the end Python is the king of readability and maintenance.

Both languages were able to handle OOP concepts like encapsulation, inheritance, and polymorphism, but the implementation varied greatly. C++'s encapsulation is vastly better than Python's. This is because C++ physically prevents other parts of the code from accessing protected or private variables while python relies on convention. In python an underscore before a variable suggests privacy but can't strictly enforce it. For inheritance and polymorphism Python's implementation was easy, allowing for similar sunglasses to be in the same list without any complex syntax. Along with the `isinstance()` method allowing for easy filtering. In C++ this was the hardest part of the implementation. To store similar classes together we needed to use pointers in a vector. In the end C++ offers better control over OOP principals, while python offers more flexible usage.

Finally error handling was Python's domain. Try-except blocks were crucial for error handling, this provided a quick and robust way to find errors, display them, and keep running. C++ struggled with this, especially when needing to perform input validation. C++ needed specific `in.clear()` and `in.ignore()` methods to clean up the input stream. While C++ does have try-catch blocks they are not very effective for user inputs. Python is much more user friendly and requires less defensive programming.

The only part of the project Python struggled with was with encapsulation. For this project its readability and ease of implementation vastly outperformed C++. Personally, Python was the better choice for this project, but C++ should be considered for a larger program where being explicit with objects and memory is more important.

AI prompts:

Google

how to limit a object's declaration parameters to a few option in python

python

```
from typing import Literal

class User:
    def __init__(self, role: Literal["admin", "user", "guest"]):
        self.role = role

# Valid
u = User("admin")
# Invalid - type checker will flag this
# u = User("superuser")
```

Google

how to clear terminal in python

python

```
import os
import time

def clear_terminal():
    # Check the operating system name
    if os.name == 'nt':
        # Command for Windows
        _ = os.system('cls')
    else:
        # Command for macOS and Linux (posix)
        _ = os.system('clear')

# Example usage:
print("This text will be cleared in 3 seconds...")
time.sleep(3)
clear_terminal()
print("The terminal has been cleared.")
```

Give me a code snippet that makes the item viewing screen neater

To make the item viewing screen look like professional inventory software, we can use **f-string alignment** to create a structured table. This ensures that columns stay perfectly aligned regardless of how long the item names are.

I've also added a **border box** and a **dynamic header** to make it stand out visually.

Neat Viewing Snippet

Replace your current `display_filtered_items` method with this version:

Python



```
def display_filtered_items(self, dept, sect):
    while True:
```

How can I better use Inheritance in my python program?
please give an example for camping such as weight and waterproof

In Python, inheritance allows you to create a "Base Class" that contains all the common logic (like price, name, and quantity) and "Child Classes" that add specific features.

Currently, your `Item` class is doing too much heavy lifting by trying to be everything at once. By using inheritance, you can make the code cleaner and more specialized.

The Strategy

1. **Base Class (`Item`)**: Handles basic inventory logic.
2. **Child Class (`CampingItem`)**: Inherits from `Item` and adds `weight` and `is_waterproof`.
3. **Child Class (`SportsItem`)**: Inherits from `Item` and might add `brand` or `material`.

Please generate a make file for the following c++ program structure: Item.cpp Item.h menu.h menu.cpp and InventorySystemMain.cpp

Show thinking ▼

Here is the Makefile specifically tailored to your file structure (`Item.cpp` , `Item.h` , `menu.h` , `menu.cpp` , `InventorySystemMain.cpp`).

I have ensured the casing matches exactly (e.g., lowercase `menu.cpp`) to prevent errors on Linux or macOS.

Makefile

Makefile

